KOENIG

step forward



Multithreading and Concurrency

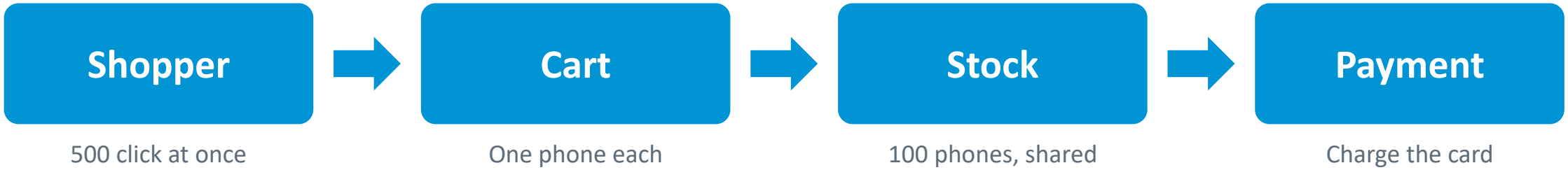
Core Java (Advanced)

Multithreading and Concurrency

1. Threads, Runnable, Callable, Future
2. Shared state, and synchronized
3. Atomic variables
4. ExecutorService and CompletableFuture
5. Concurrent collections
6. Virtual threads

Demo Scenario: ShopKart Flash Sale

Module 4 handled failures one order at a time. Now 500 shoppers press Buy together. ShopKart sells 100 phones in a flash sale, and 500 shoppers click Buy at the same moment. Every shopper is a task, and all of them share one stock count.



What can go wrong

- **Lost updates**
- **Overselling**
- **Slow I/O calls**

Why Concurrency?

- **Responsiveness:** the UI or web server keeps answering while work runs.
- **Throughput:** use all CPU cores, not just one.
- **Waiting:** while one task waits for I/O, another can run.

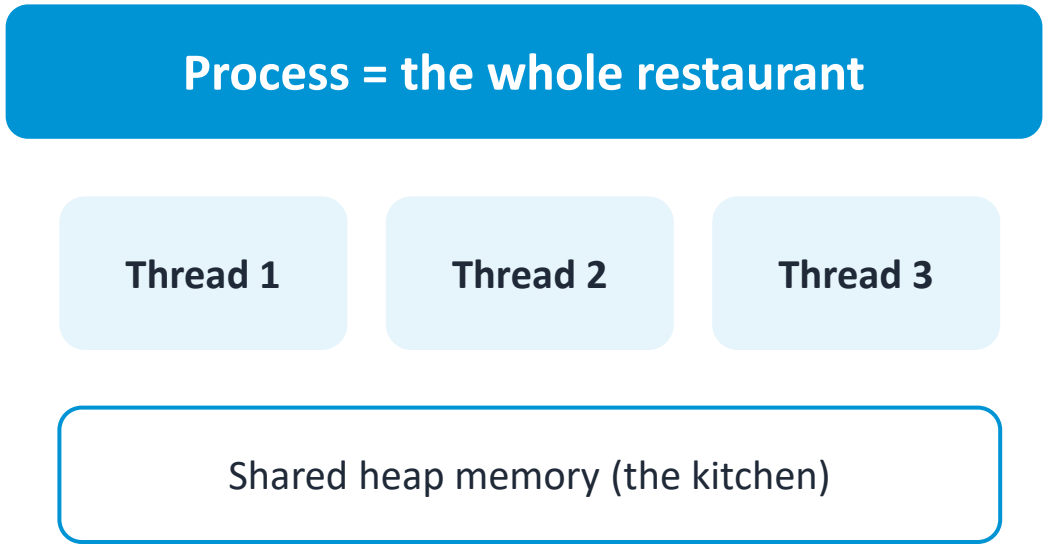
One cook
one order at a time

Many cooks
orders in parallel, but they share one stove

Shared stove
needs rules, or the soup burns

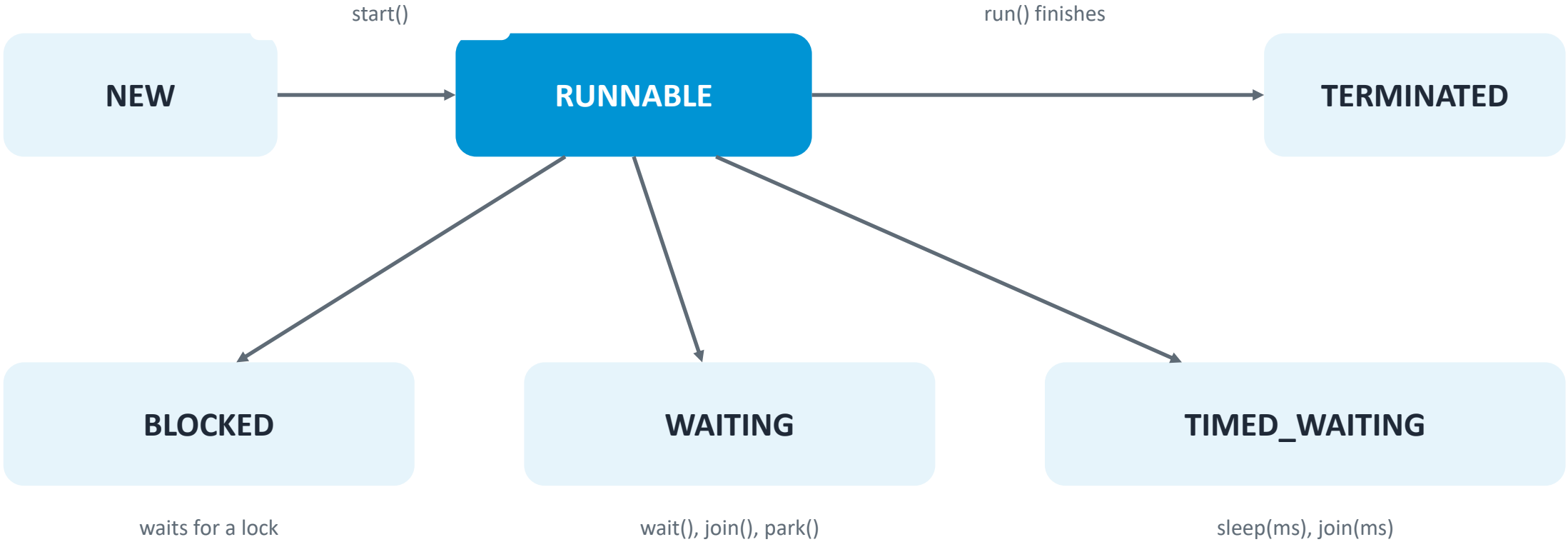
Concurrency = dealing with many things; parallelism = doing many at once

Processes and Threads



- **Thread:** an independent path of execution (a cook).
- **Own:** its stack and local variables.
- **Shared:** the heap, so objects and static fields.
- **Every Java program** starts with the main thread.

The Thread Lifecycle



RUNNABLE means ready or running; the OS scheduler picks who runs

Creating Threads with Runnable

- **Runnable:** a job with no result.
- **Thread:** the worker that runs the job.
- **start()** creates the new thread.
- **run()** directly is just a normal method call!
- **join()** waits for a thread to finish.

```
Runnable job = () -> System.out.println(
    "Hi from " + Thread.currentThread().getName());

Thread t = new Thread(job, "cook-1");
t.start();       // new thread runs the job
t.join();        // main waits for it
System.out.println(t.getState());
// State: TERMINATED
```

Never call run() yourself - always call start()

Demo: Threads and Runnable

What we build

- **Runnable:** a named cook thread that prints its own name.
- **start vs run:** calling run() directly stays on the main thread.
- **Lifecycle:** NEW before start, TERMINATED after join.

How the concept helps

- **A thread** is an independent path of execution.
- **start()** is what really creates the new thread.
- **join()** lets one thread wait for another to finish.

Callable and Future

- **Callable<V>:** like Runnable, but returns a value and can throw.
- **Future<V>:** a receipt for a result that will exist later.
- **get()** blocks until the result is ready.
- **isDone()** checks without waiting.
- **cancel()** tries to stop the task.

```
ExecutorService pool =
    Executors.newSingleThreadExecutor();

Callable<Integer> job = () -> {
    Thread.sleep(500);
    return 42;
};

Future<Integer> receipt = pool.submit(job);
System.out.println("Working...");
System.out.println(receipt.get());   // waits: 42
pool.shutdown();
```

Demo: Callable and Future

What we build

- **Price lookup:** a Callable returns a value after a short delay.
- **Future:** isDone, get and cancel on the receipt.
- **Failure:** an exception inside the task comes out of get().

How the concept helps

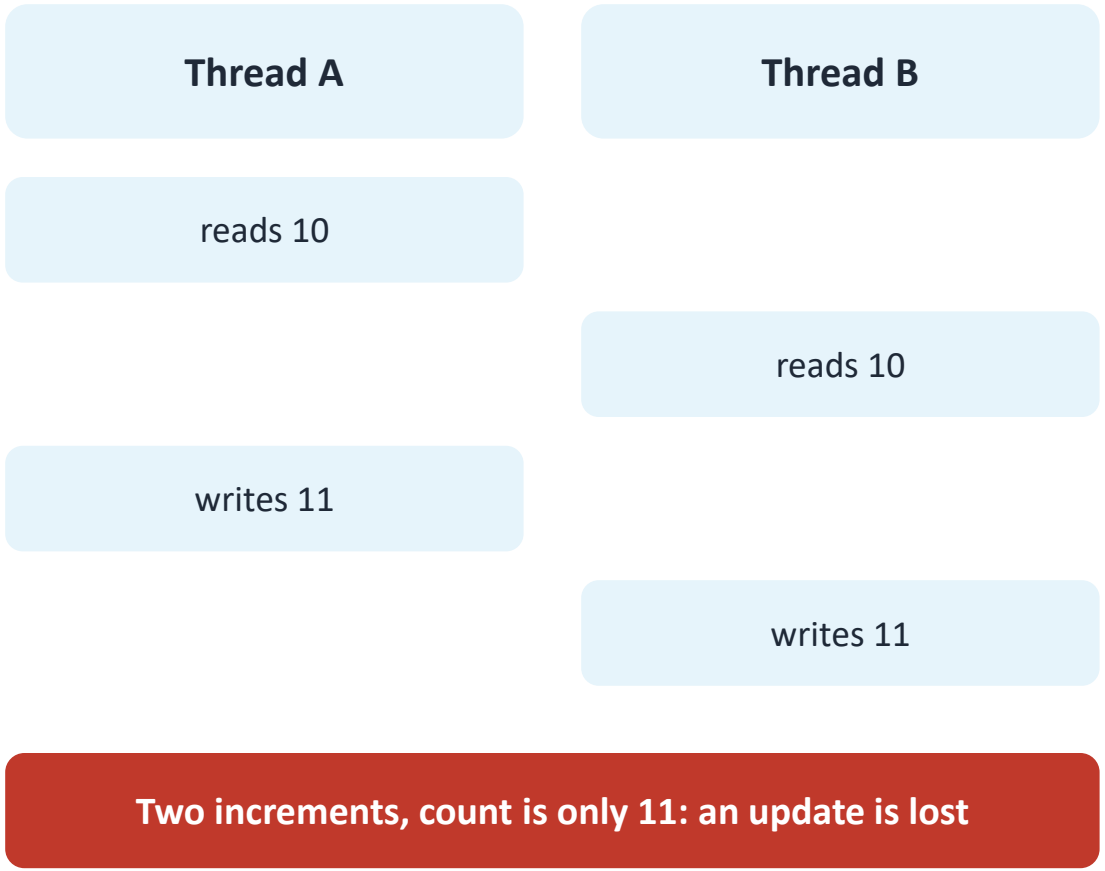
- **Callable** returns a result and may throw.
- **Future** is the receipt for work that finishes later.
- **get()** waits only when we ask for the answer.

Shared Mutable State

- **Shared:** more than one thread can reach it.
- **Mutable:** it can change.
- **Both together** = danger. Remove one and you are safe.
- **count++** is read, add, write: three steps.

```
private int count = 0;

void increment() {
    count++;   // NOT atomic
}
```



Memory Visibility and volatile

- **Problem:** a thread may keep a stale copy in its cache.
- **Without help,** one thread may never see another's write.
- **volatile** forces reads and writes to go to main memory.
- **Good for:** simple flags, like stop.

```
class Worker implements Runnable {
    private volatile boolean running = true;

    public void run() {
        while (running) {
            doWork();
        }
    }

    void stop() {
        running = false;   // seen by the worker
    }
}
```

synchronized and Intrinsic Locks

- **Every object** has a built-in lock (monitor).
- **synchronized** lets one thread in at a time.
- **Others wait** in the BLOCKED state.
- **Keep** the locked section short.

```
class Counter {
    private long count;

    synchronized void increment() { // locks "this"
        count++;
    }

    synchronized long get() {
        return count;
    }
}

// or lock a specific object:
synchronized (account) {
    account.add(100);
}
```

Analogy: one key to the bathroom - whoever holds it, others queue

ReentrantLock: Explicit Locks

- **lock() / unlock():** you manage it yourself.
- **Always** unlock in finally.
- **tryLock():** try without waiting forever.
- **Fairness** option: longest waiter first.
- **Reentrant:** the same thread can lock again.

```
private final ReentrantLock lock = new ReentrantLock();

void increment() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock();   // always runs
    }
}

if (lock.tryLock(1, TimeUnit.SECONDS)) {
    try { count++; } finally { lock.unlock(); }
}
```

ReadWriteLock: Many Readers

Read lock
many threads at the same time

Write lock
exactly one thread, no readers

- **Best when** reads are much more common than writes.
- **Example:** a price list read all day, updated once an hour.

```
ReadWriteLock rw = new ReentrantReadWriteLock();

String read() {
    rw.readLock().lock();
    try { return data; }
    finally { rw.readLock().unlock(); }
}

void write(String v) {
    rw.writeLock().lock();
    try { data = v; }
    finally { rw.writeLock().unlock(); }
}
```

Atomic Variables

- **AtomicInteger, AtomicLong,** AtomicBoolean, AtomicReference.
- **No lock:** uses a CPU compare-and-set.
- **LongAdder:** faster for counters with many threads.
- **Only for** one variable at a time.

```
AtomicLong hits = new AtomicLong();
hits.incrementAndGet();          // thread-safe ++
hits.addAndGet(5);
hits.compareAndSet(6, 100);      // only if value is 6

LongAdder adder = new LongAdder();
adder.increment();
adder.add(5);
long total = adder.sum();        // add up the cells
```

Compare-and-set: change it only if nobody else changed it first

Choosing a Locking Tool

1. **synchronized**
   Simplest. Default choice for guarding several fields together.

2. **ReentrantLock**
   Need tryLock, timeouts or fairness. Must unlock in finally.

3. **AtomicLong**
   One shared number or reference, low to medium contention.

4. **LongAdder**
   Hot counter, many writers, read rarely.

Best of all: do not share mutable state at all

Demo: synchronized, Locks and Atomics

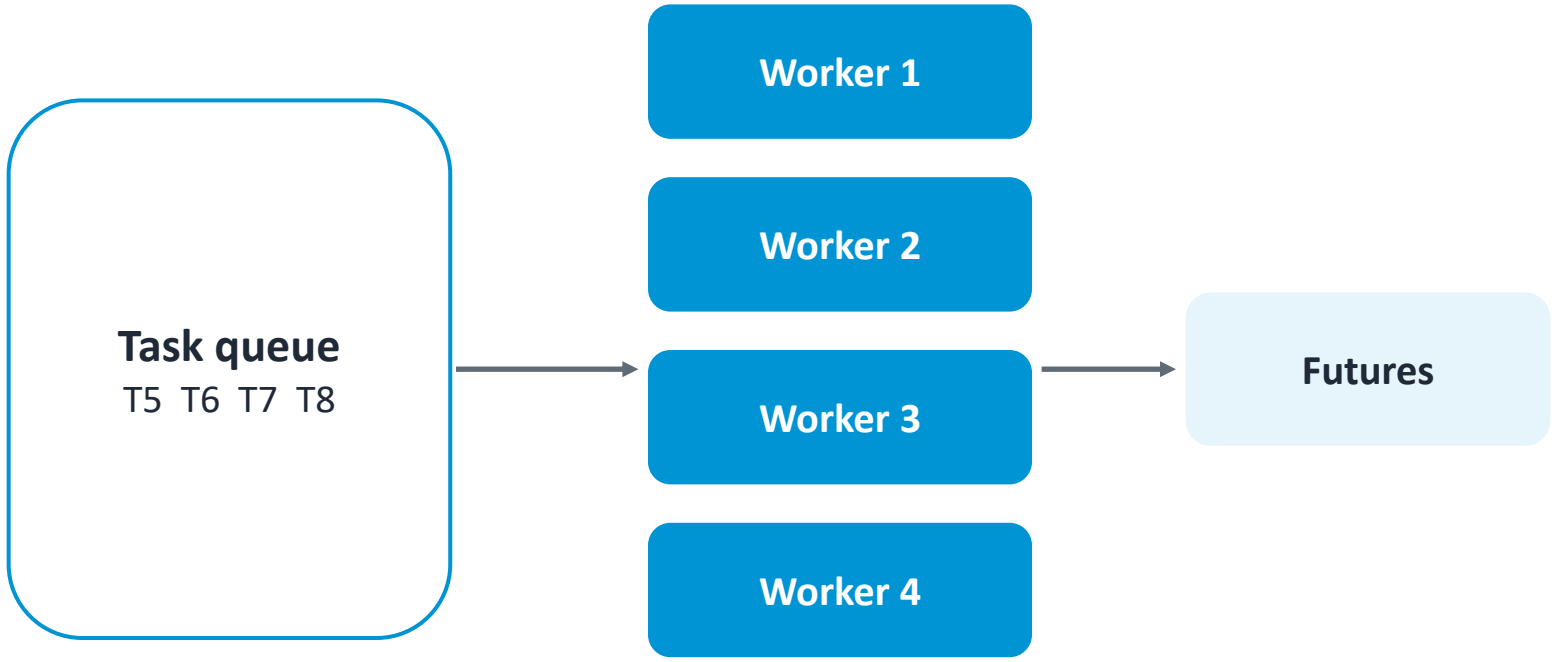
What we build

- **One counter, five fixes:** synchronized, ReentrantLock, tryLock, AtomicLong, LongAdder.
- **ReadWriteLock:** many readers of the price list, one writer.
- **Compare-and-set:** change a value only if nobody else did.

How the concept helps

- **A lock** lets one thread in at a time.
- **Atomics** protect one variable without a lock.
- **The right tool** depends on what we share and how often.

ExecutorService: Thread Pools



- **Reuse** threads: creating them is costly.
- **Limit** how many run at once.
- **Queue** extra tasks wait their turn.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> doWork());
List<Future<Long>> all = pool.invokeAll(tasks);
```

Graceful Shutdown

- **shutdown():** no new tasks, finish queued ones.
- **awaitTermination():** wait with a timeout.
- **shutdownNow():** interrupt running tasks.

```
pool.shutdown();
try {
    if (!pool.awaitTermination(10, TimeUnit.SECONDS)) {
        pool.shutdownNow();      // give up, interrupt
        pool.awaitTermination(5, TimeUnit.SECONDS);
    }
} catch (InterruptedException e) {
    pool.shutdownNow();
    Thread.currentThread().interrupt();
}

// Java 19+: close() does the same wait
try (ExecutorService p = Executors.newFixedThreadPool(2)) {
    p.submit(() -> work());
}
```

Demo: Thread Pools

What we build

- **Fixed pool:** four workers process twenty orders.
- **invokeAll:** run every task and read every result.
- **Shutdown:** shutdown and awaitTermination, or try-with-resources.

How the concept helps

- **A pool** reuses threads instead of creating new ones.
- **Shutdown** stops forgotten pools from keeping the JVM alive.

CompletableFuture

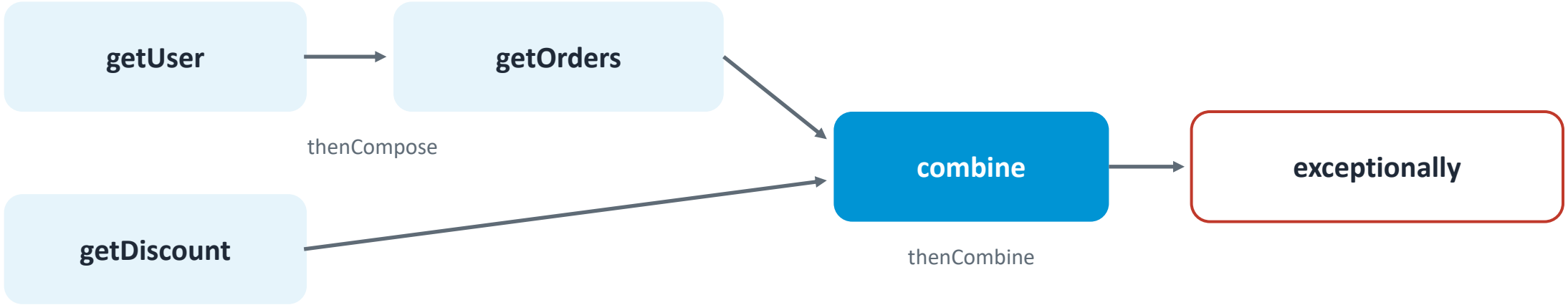
- **Future** but you can attach next steps.
- **supplyAsync:** start work, get a value.
- **thenApply:** transform the result.
- **thenAccept:** consume it.
- **join()** wait (unchecked exceptions).

```
CompletableFuture<String> f =
    CompletableFuture
        .supplyAsync(() -> fetchName())      // async
        .thenApply(name -> name.toUpperCase())
        .thenApply(n -> "Hello " + n);

f.thenAccept(System.out::println);
String text = f.join();
```

No blocking between steps: each runs when the previous completes

Composing Async Calls




```
getUser(7)
    .thenCompose(u -> getOrders(u))       // depends on u
    .thenCombine(getDiscount(7),          // independent
                 (orders, d) -> orders + " / " + d + "%")
    .exceptionally(ex -> "fallback")      // on any failure
    .thenAccept(System.out::println);
```

Demo: CompletableFuture

What we build

- **Pipeline:** supplyAsync then thenApply, with no blocking in between.
- **Composing:** thenCompose for a dependent call, thenCombine for two independent calls.
- **Failure:** exceptionally gives a fallback when a step fails.

How the concept helps

- **Async steps** run as soon as the previous one completes.
- **thenCombine** overlaps independent calls to save time.
- **exceptionally** keeps one failure from breaking the whole chain.

ConcurrentHashMap

- **Thread-safe map** without locking the whole map.
- **No null** keys or values.
- **Atomic methods:** merge, compute, putIfAbsent.
- **HashMap** is NOT safe across threads.
- **Avoid** check-then-put: it races.

```
Map<String, Integer> words = new ConcurrentHashMap<>();

// broken: two steps, can lose a count
// if (words.containsKey(w)) words.put(w, words.get(w)+1);

// correct: one atomic step
words.merge("java", 1, Integer::sum);

words.computeIfAbsent("loom", k -> 0);
```

CopyOnWriteArrayList

How it works

Every write copies the whole array. Readers keep using the old snapshot.

Good for

Many reads, very few writes: listener lists, config lists.

```
List<String> listeners = new CopyOnWriteArrayList<>();
listeners.add("audit");

for (String l : listeners) {   // safe, never throws
    listeners.add("late");     // does not break the loop
}
```

Bad for

large lists with frequent writes

Demo: Concurrent Collections

What we build

- **ConcurrentHashMap:** check-then-put loses counts, merge does not.
- **CopyOnWriteArrayList:** add while looping, no exception.

How the concept helps

- **Concurrent maps** give atomic one-step updates.
- **Copy on write** suits lists that are read often and rarely changed.

Virtual Threads (JDK 21)

- **Lightweight:** you can have millions.
- **Managed by the JVM,** not the OS.
- **When blocked,** the carrier thread is freed.
- **Great for** blocking I/O, such as web requests.
- **Not for** CPU-heavy work. Do not pool them.

```
try (ExecutorService ex =
        Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        ex.submit(() -> {
            Thread.sleep(100);   // cheap to block
            return "done";
        });
    }
}   // close() waits for all tasks

Thread.startVirtualThread(() -> work());
```

One virtual thread per task: write simple blocking code, get scale

Platform vs Virtual Threads

Platform thread

- Maps 1:1 to an OS thread
- About 1 MB of stack
- Thousands at most
- Pool and reuse them
- Good for CPU work

Virtual thread

- Scheduled by the JVM
- Few KB, grows on demand
- Millions are fine
- Create one per task
- Good for I/O waiting

Demo: Virtual Threads

What we build

- **1000 slow calls:** a pool of 50 platform threads versus one virtual thread per task.
- **Check:** Thread.isVirtual() on main and on a task.

How the concept helps

- **Virtual threads** make waiting cheap, so blocking code scales.
- **No pooling** is needed: create one per task.
- **CPU-heavy work** still belongs on a sized platform pool.

Demo: Lab, the Flash Sale

What we build

- **The rush:** 500 shoppers on a pool of 8 threads fight for 100 phones.
- **No overselling:** an AtomicInteger with a compare-and-set loop.
- **Revenue:** a LongAdder, and a final report that always adds up.

How the concept helps

- **Atomic stock** makes every sale all-or-nothing.
- **The answer** is the same on every run: 100 sold, 400 rejected.
- **Reuse** brings together pools, atomics and futures.

Key Takeaways

1. **Share less**
   Immutable data and local variables need no locks.

2. **Know the tools**
   volatile, synchronized, atomics, locks.

3. **Use pools**
   ExecutorService, and shut it down.

4. **Compose**
   CompletableFuture and concurrent collections.

5. **Go virtual**
   Virtual threads for blocking I/O.

6. **Test hard**
   Run many times; read thread dumps.



Happy Learning :)